

Chat TCP

Aplicação de chat em tempo real com sockets TCP

Disciplina: Redes de Computadores

Instituto Federal de São Paulo — Campus São Carlos

Integrantes:

- Guilherme Perez Gomes - SC3044653
- Maria Clara Passareli Alves - SC3044289
- Mateus Ciffoni - SC3044645

1. Descrição da Aplicação

O Chat TCP é uma aplicação de mensagens em tempo real implementada com sockets TCP em Python. A aplicação segue o modelo cliente-servidor: um processo servidor central gerencia as conexões e encaminha as mensagens entre os participantes; cada participante executa um processo cliente que se conecta ao servidor para enviar e receber mensagens.

A aplicação suporta múltiplos clientes simultaneamente por meio de threads — cada cliente conectado recebe uma thread dedicada no servidor. Os participantes escolhem um apelido ao entrar e podem criar ou ingressar em salas de conversa. As mensagens são transmitidas apenas para os participantes da mesma sala.

Tecnologias utilizadas:

- Python 3.8 ou superior
- Módulo socket (biblioteca padrão)
- Módulo threading (biblioteca padrão)
- Protocolo TCP/IP

Arquitetura:

O servidor mantém um dicionário de salas onde cada chave tem o nome da sala e o valor é uma lista de clientes conectados naquela sala. Quando um cliente envia uma mensagem, o servidor realiza um broadcast para todos os outros clientes da mesma sala. O acesso ao dicionário é protegido por um Lock para evitar condições de corrida entre as threads.

2. Estrutura do Projeto

```
chat-socket/  
| server.py # Servidor TCP
```

```
| client.py # Cliente TCP
| README.md # Instruções e link do vídeo
| docs/
|- manual.pdf # Este documento
```

Descrição dos arquivos:

server.py: inicializa o socket TCP, aguarda conexões de clientes, cria uma thread por cliente e gerencia salas, apelidos, broadcast de mensagens e comandos especiais.

client.py: conecta ao servidor, realiza o fluxo de entrada (apelido e sala) e mantém duas threads em execução — uma para receber mensagens do servidor e outra para ler o input do usuário e enviar ao servidor.

3. Manual do Usuário

3.1 Requisitos

- Python 3.8 ou superior instalado
- Nenhuma dependência externa — apenas a biblioteca padrão do Python
- Para testes em rede local: servidor e clientes devem estar na mesma rede e o HOST em client.py deve ser alterado para o IP da máquina que executa o servidor

3.2 Como executar o servidor

Abra um terminal e execute:

```
python server.py
```

O servidor exibirá a mensagem:

```
[SERVIDOR] Escutando em 0.0.0.0:5555 ...
[SERVIDOR] Aguardando clientes. Pressione Ctrl+C para encerrar.
```

O servidor ficará em execução até que seja encerrado com Ctrl+C. Mantenha este terminal aberto enquanto os clientes estiverem conectados.

3.3 Como executar o cliente

Em outro terminal (ou em outra máquina da rede), execute:

```
python client.py
```

O cliente solicitará o apelido e a sala:

```
[CLIENTE] Conectado ao servidor 127.0.0.1:5555

Digite seu apelido: Gui

Bem-vindo, Gui!
Nenhuma sala disponível no momento.
Digite o nome da sala para entrar (ou crie uma nova): geral
```

```
Você entrou na sala #geral. Digite /ajuda para ver os comandos.
```

Repita este passo em outros terminais para simular múltiplos participantes.

3.4 Enviando mensagens

Após entrar em uma sala, basta digitar o texto e pressionar Enter. A mensagem será enviada ao servidor e distribuída para todos os outros participantes da mesma sala no seguinte formato:

```
[#geral] Gui: Olá pessoal!
```

3.5 Comandos disponíveis

Os comandos são palavras-chave iniciadas com / que realizam ações especiais no servidor.

```
/ajuda exibe a lista de comandos disponíveis  
/salas lista as salas ativas e seus participantes  
/trocar <sala> muda para a sala informada (cria se não existir)  
/sair desconecta do servidor
```

Exemplo de uso do comando /trocar:

```
/trocar suporte  
Você entrou na sala #suporte.
```

Exemplo de uso do comando /salas:

```
/salas  
Salas disponíveis:  
#geral (2 pessoa(s)): Maria, Pedro  
#suporte (1 pessoa(s)): Gui
```

3.6 Encerrando o cliente

Para encerrar a conexão, utilize o comando /sair ou pressione Ctrl+C. O servidor será notificado e os demais participantes da sala receberão uma mensagem informando que o usuário saiu.

3.7 Encerrando o servidor

Para encerrar o servidor, pressione Ctrl+C no terminal onde ele está em execução. Todas as conexões ativas serão encerradas.

4. Exemplo de Sessão Completa

Terminal 1 — Servidor:

```
$ python server.py  
[SERVIDOR] Escutando em 0.0.0.0:5555 ...  
[SERVIDOR] Aguardando clientes. Pressione Ctrl+C para encerrar.  
  
[SERVIDOR] Nova conexão: ('127.0.0.1', 52301)  
[SERVIDOR] ('127.0.0.1', 52301) entrou como 'Gui'  
[SERVIDOR] 'Gui' entrou na sala 'geral'  
[SERVIDOR] Nova conexão: ('127.0.0.1', 52302)
```

```
[SERVIDOR] ('127.0.0.1', 52302) entrou como 'Maria'
[SERVIDOR] 'Maria' entrou na sala 'geral'
[SERVIDOR] [#geral] Gui: ola Maria!
[SERVIDOR] [#geral] Maria: oi Gui!
```

Terminal 2 — Cliente Gui:

```
$ python client.py
[CLIENTE] Conectado ao servidor 127.0.0.1:5555

Digite seu apelido: Gui

Bem-vindo, Gui!
Nenhuma sala disponível no momento.
Digite o nome da sala para entrar (ou crie uma nova): geral

Você entrou na sala #geral. Digite /ajuda para ver os comandos.
[Maria entrou na sala #geral]
ola Maria!
[#geral] Maria: oi Gui!
```

Terminal 3 — Cliente Maria:

```
$ python client.py
[CLIENTE] Conectado ao servidor 127.0.0.1:5555

Digite seu apelido: Maria

Bem-vindo, Maria!
Salas disponíveis:
#geral (1 pessoa(s)): Gui
Digite o nome da sala para entrar (ou crie uma nova): geral

Você entrou na sala #geral. Digite /ajuda para ver os comandos.
[#geral] Gui: ola Maria!
oi Gui!
```

5. Repositório

Código fonte:

<https://github.com/guipiegomes/sockets-rc1-py>

Vídeo no youtube:

https://www.youtube.com/watch?v=BFVrJIrf_Y0